

METROPOLIA UAS · TX00-WEB-EN · GROUP 4

Migrant Hub

SPRINT 2

A React frontend and an Express API

Pratham Arora · Prabhleen Kaur · Sajib Das · Yun Wang · Sehinder Singh

SPRINT GOAL

One goal, Two halves, One contract

Build the React frontend and the Express backend separately, agree the API between them and deliberately do not connect them. That connection is Sprint 3.

FRONTEND · REACT + VITE

Pratham Arora, Prabhleen Kaur, Sajib Das

Nine pages on a shared shell, styled from one set of CSS variables, running on mock data written in the contract's shapes.

BACKEND · EXPRESS + MONGODB

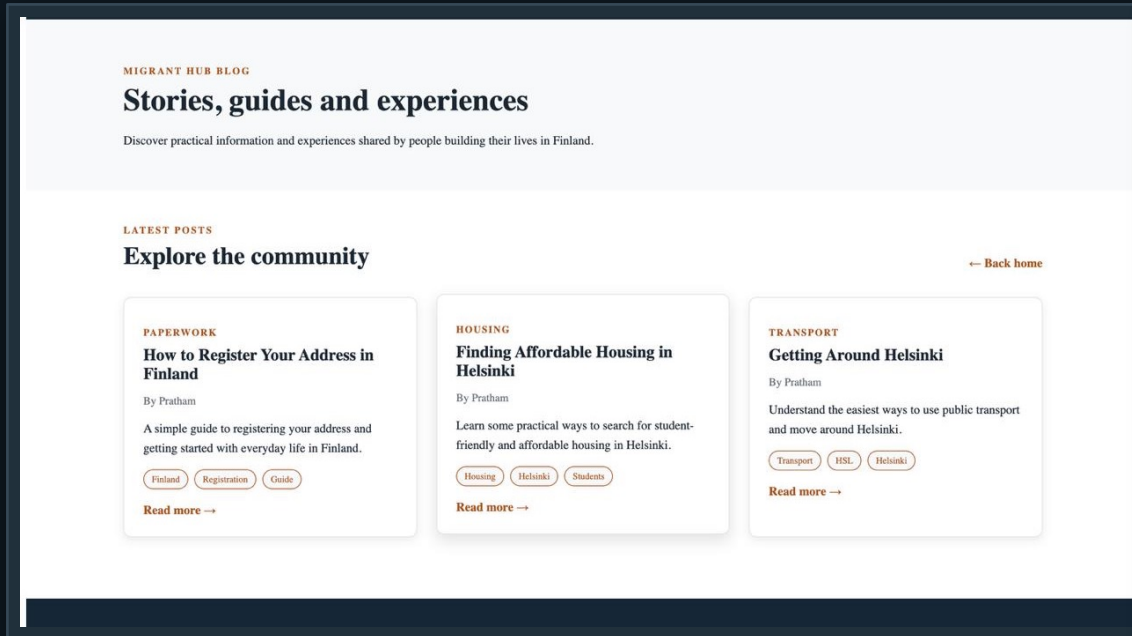
Yun Wang, Sehinder Singh

Split by layer rather than by feature - Yun owns the server and the routes, Sehinder owns the models and the database.

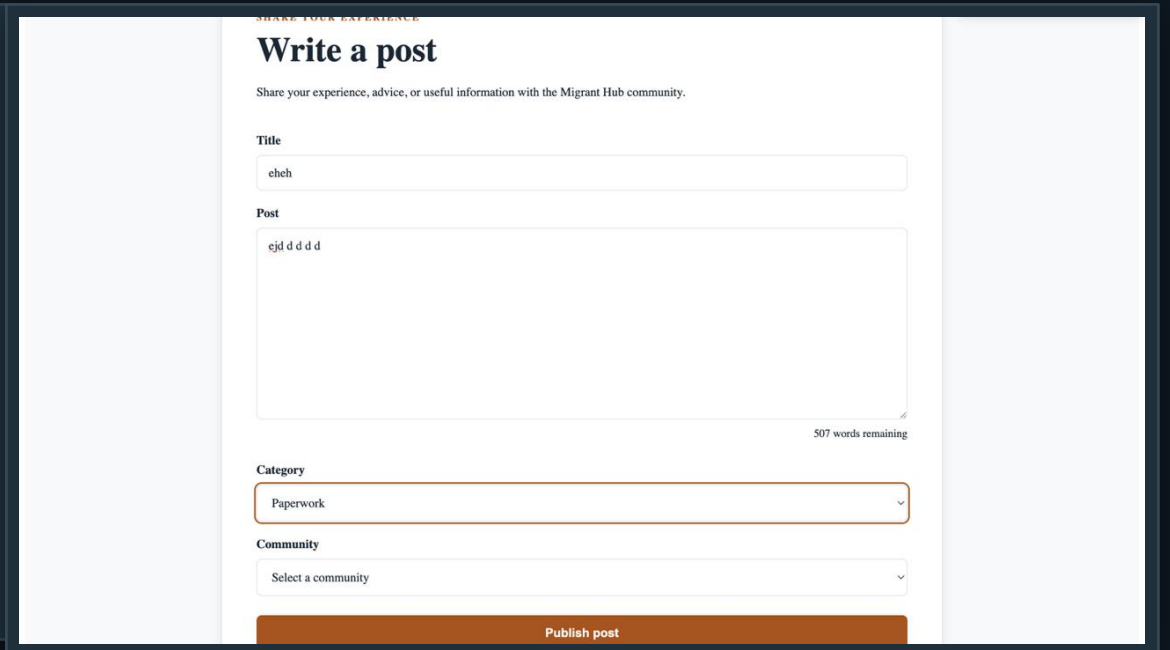
THE MEETING POINT

One API contract, committed to the repo on day two: every endpoint, every field name, every type, every status code. The frontend wrote its mock data in exactly those shapes. The backend returned exactly those shapes. Neither side waited for the other for two weeks.

Blog & Publishing



Blog list - post cards with category, author and tags



Write a post - live word counter, category and community

- Blog list page: post cards showing category, title, author, summary and tags
- Write a post form: controlled inputs with a live counter enforcing the 512-word limit
- Category and community selectors built from the agreed category list
- Everything runs on mock data written in the shapes the API contract promises

Shell, routing and homepage

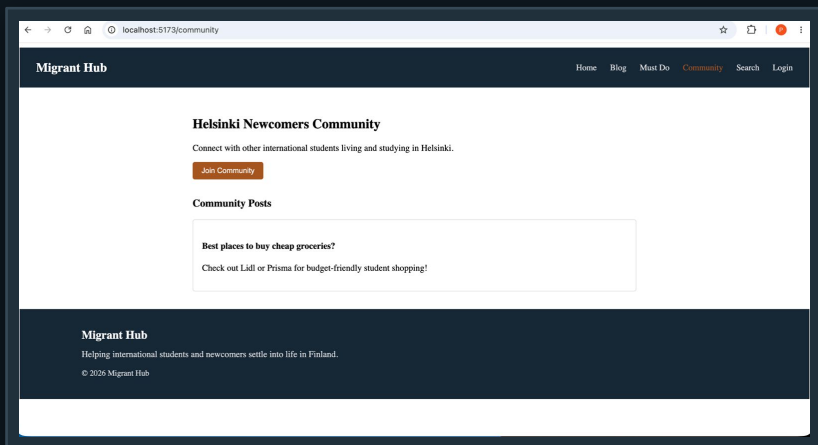
Set up the Vite React project and the folder structure the whole team builds in

Navbar and footer as shared components, used on every page

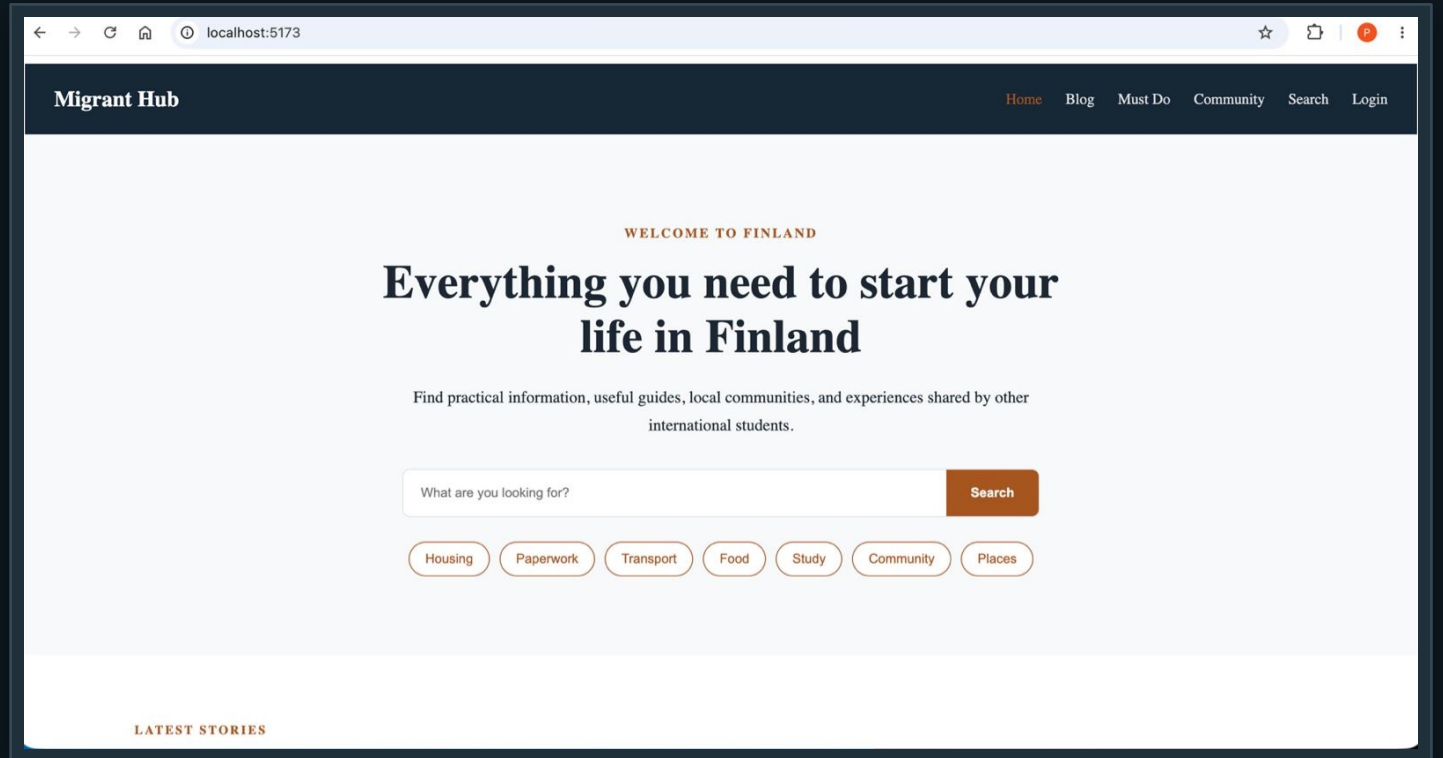
Routing, so every navbar link opens a real page, this unblocked the other two on day one

Shared CSS variables, so three people style from one set of values

Homepage: hero, search box and the seven category chips

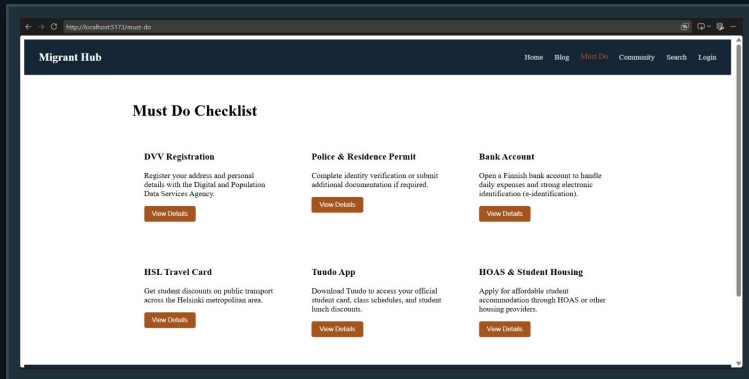


Community page

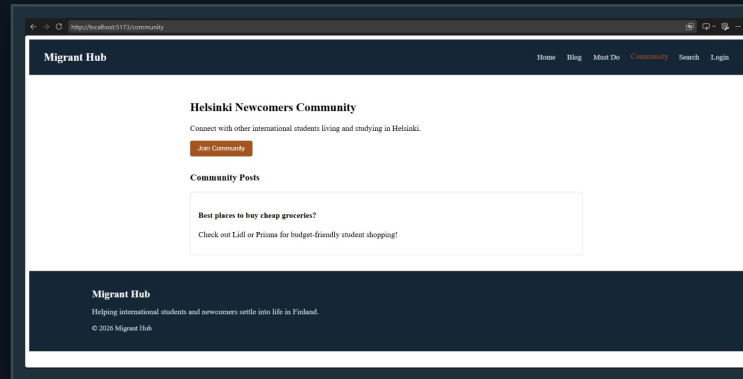


Homepage - hero, search and the seven categories from the contract, Places included

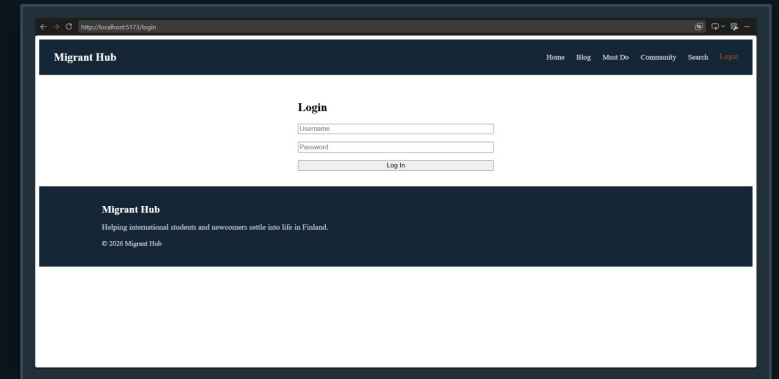
Must Do, community & login



Must Do checklist



Community page



Login

Must Do checklist: the six official steps a newcomer has to complete, each with a detail page

Community page with the join action and community posts

Login page the form and the flow only; no real authentication this sprint, by design

Scrum Master for Sprint 2: ran the daily scrums and kept the log in the repo

Those six Must Do items are the same six that sit in the seeded database, built by two people who never read each other's code. That is the API contract doing its job.

Server & API

Express server in MVC structure: routes, controllers, middleware

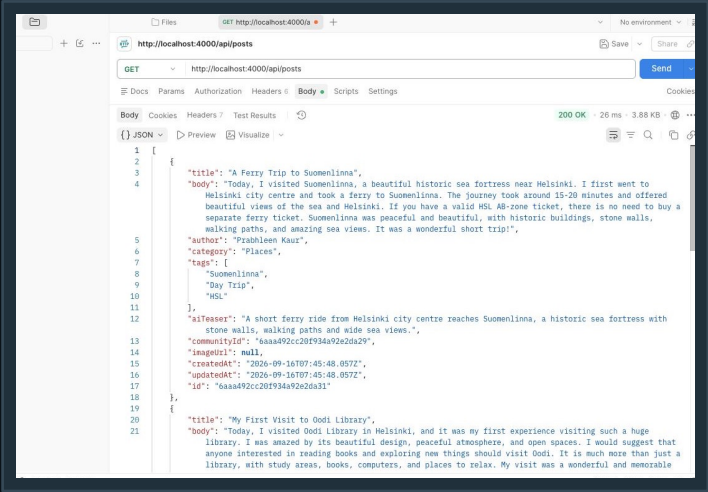
Full create, read, update and delete for posts, communities and Must Do items

Validation middleware rejects bad input with 400 before it reaches the controller

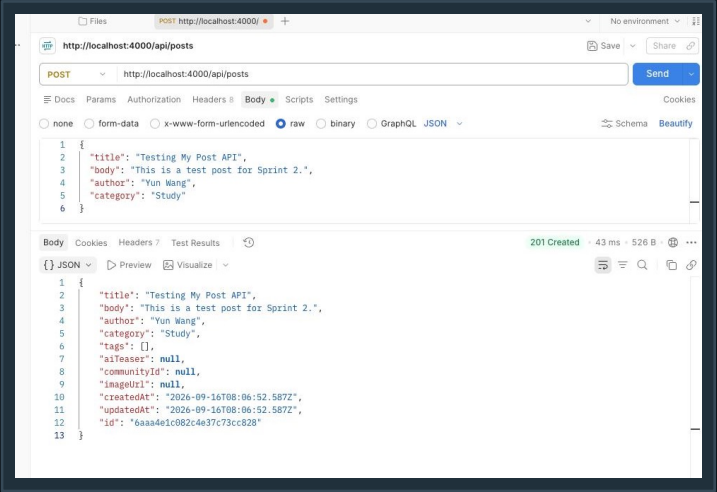
Error handler returns JSON for a missing record, never an HTML crash page

One unified error shape across every endpoint , a deliberate fix on the inconsistency in our own Tour API lab

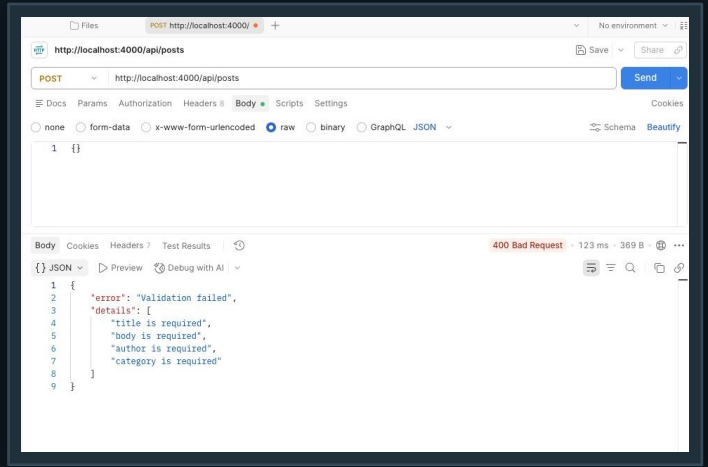
Postman collection covering every endpoint, including the failures



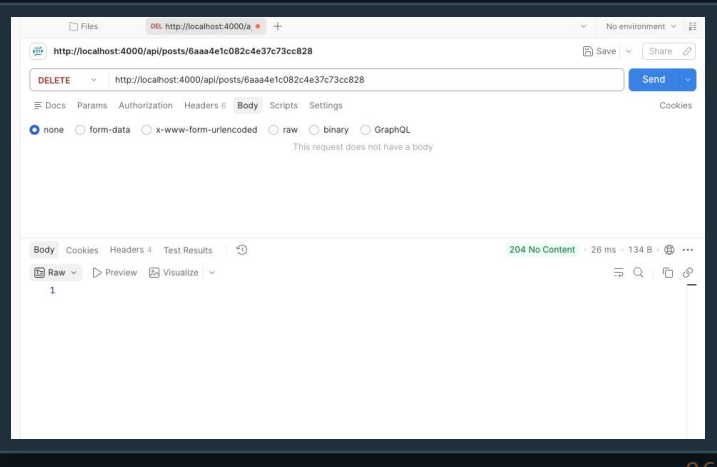
GET /api/posts - 200, seeded data



POST /api/posts - 201 Created



Empty body - 400 with field errors



DELETE - 204, no content

The Data Layer

```

1 require("dotenv").config();
2
3 const mongoose = require("mongoose");
4 const connectDB = require("./config/db");
5
6 const Post = require("./models/postModel");
7 const Community = require("./models/communityModel");
8 const MustDo = require("./models/mustDoModel");
9
10 const communities = [
11   {
12     name: "Finland Newcomers",
13     description:
14       "A community for international students and newcomers in Finland to share experiences, ask questions and find useful advice.",
15     memberCount: 42,
16   },
17   {
18     name: "Indians in Helsinki",
19     description:
20       "Students and newcomers from India living in the Helsinki area - food, festivals, groceries and practical help.",
21     memberCount: 18,
22   },
23 ];
24
25 const mustDoItems = [

```

```

shwindersingh@shwinders-MacBook-Air migrant-hub-backend % npm run seed
> migrant-hub-backend@1.0.0 seed
> node seed.js
o injected env (2) from .env // tip: * custom filepath { path: 'custom/path/.env' }
MongoDB connected
Old data removed
Communities: 2
Must Do items: 6
Posts: 4
Seeding complete
shwindersingh@shwinders-MacBook-Air migrant-hub-backend %

```

One command fills the database with our own Sprint 1 content

Collection name	Properties	Storage size	Data size	Documents	Avg. document size	Indexes	Total index size
communities	-	36.86 kB	499.00 B	2	249.00 B	2	57.34 kB
mustdos	-	36.86 kB	5.02 kB	6	837.00 B	2	57.34 kB
posts	-	36.86 kB	3.63 kB	4	906.00 B	1	36.86 kB

MongoDB Atlas - three collections, 2, 6 and 4 documents

Three Mongoose schemas - Post, Community, Must Do Item with the contract's rules built in: required fields, the 512-word body limit, the category list, unique slugs

MongoDB Atlas connection, with the connection string in .env and .env gitignored from the first commit

A toJSON transform so the API returns id, not Mongo's _id the contract's field name, whatever the database stores

A seed script: npm run seed loads two communities, six Must Do items and four real posts in one command

What we would keep and what we would change

LIKED

Splitting the backend by layer rather than by feature. When MongoDB arrived, the refactor touched only the model files - Yun's controllers needed no changes at all. That was the prediction we made when we chose the split and it held.

LEARNED

Mongoose schemas and validation and something bigger: that writing the API contract before any code was what let two teams work in parallel for two weeks without ever blocking each other.

LACKED

Confidence that the two halves actually match. Both sides worked to the same contract, but nothing checked it. Keeping the frontend's mock data in the same shape as the backend's responses was manual all sprint.

LONGED FOR

To see them connected. Everything we built this sprint is only half-proven until a real request comes back from a real endpoint, which is exactly what Sprint 3 starts with.

What Sprint 3 picks up

01 **Connect the two halves**

The frontend stops reading mock data and starts calling the API. This is where we find out whether the contract really held.

02 **Real authentication**

Login and registration become real instead of a form that always succeeds.

03 **The AI feature**

At publish time, generate the teaser, suggest tags, and propose a category and community — with the API key on the server, never in the browser.

04 **Documentation and tests**

API documentation and automated tests, both deliberately left out of Sprint 2 so we did not deliver them early.

Thank you

QUESTIONS?